ECE 559 Lab Tutorial 2

ADE, Spectre, Vector Files, and NanoTime Simulation Introduction

Contents

1 Introduction

The purpose of the second lab tutorial is to help you in simulating your inverter design that you designed in the first lab tutorial. You will use HSPICE and Nanosim to simulate your design and evaluate its performance by examining the simulation results.

Upon completion of this tutorial, you should be able to:

- Simulate your schematic using Spectre
- Examine the results of your Spectre simulation
- Create a vector file for simulation
- Simulate your schematic using Spectre FX
- Extract a netlist from your schematic
- Identify critical paths using NanoTime

2 Cadence Analog Design Environment

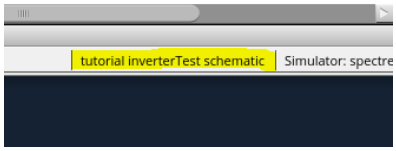
The Analog Design Environment is the place where all the simulations and the netlist extractions of your inverter design can be done. Here you can set up the type of simulation to be run with the appropriate input signals, run the simulation, and display the results in a graphical viewer.

2.1 Launch Analog Design Environment (ADE)

- If you have not opened your *InverterTest* schematic cellview, open it by selecting **File -› Open** from the drop down menu in the CIW. Select the *tutorial* library and make sure that you select the schematic view.

- In the schematic window, select **Launch -› ADE Explorer** from the drop-down menu. Select **Create New View** then click **OK,** then Click **OK** again on the next window. A new window named *Virtuoso ADE Explorer* appears. Note that your schematic has been moved into another tab in the same window.

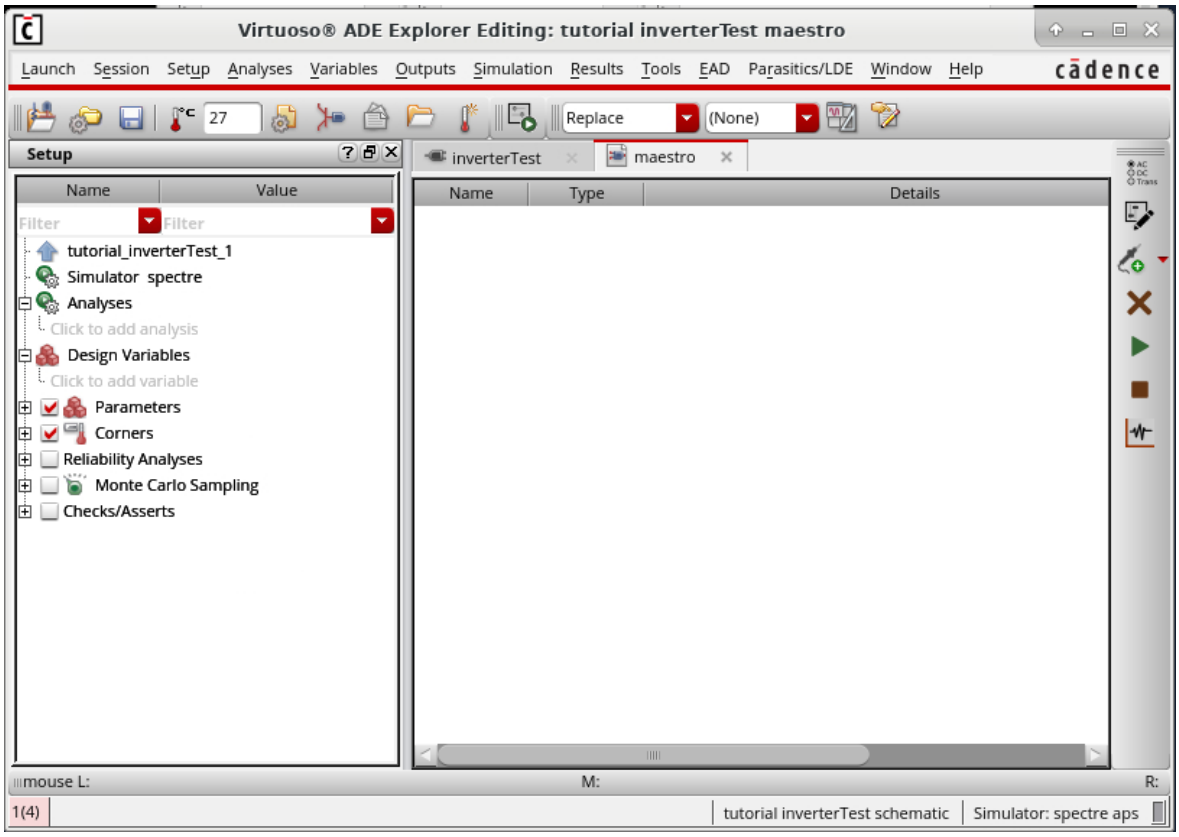
- The design you are current simulating is listed in the bottom right of the window.



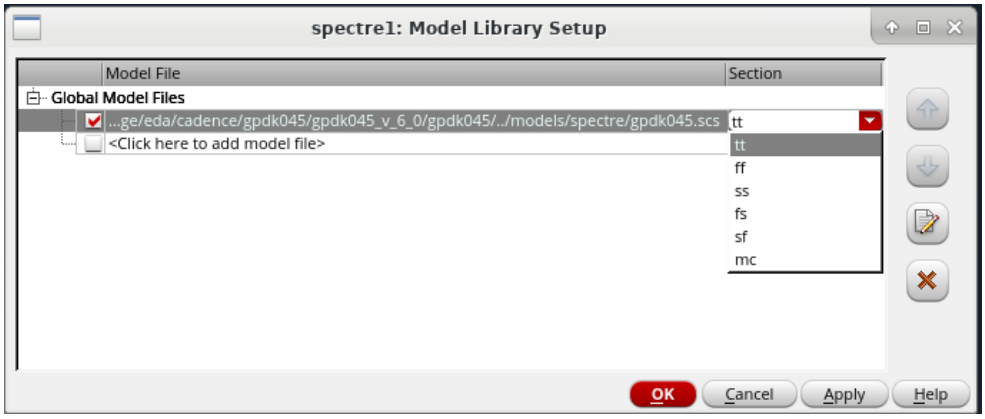
2.2 *Spectre Simulation Setup*

- To setup a Spectre simulation, select **Setup -› Simulator** from the drop down menu. A new window named *Choosing Simulator/Directory/Host* appears. Select *spectre* for the *Simulator* then click **OK**

- Select **Setup->Save Options.** The *Simulation Results Directory* field specifies where the simulation files and results are to be stored. If you need to store results in another directory or path, you can change it here. Note that results from subsequent simulation runs will overwrite the previous results unless they are to be stored in a different location. For this tutorial, just use the default path. Click **OK** to finish.



- Now select **Setup -> Model Libraries** from the drop down menu. A new window called *Model Library Setup* appears. This points to the spectre model files used for simulation. The right column for "Section" lets you choose which corner to use for simulation. Select 'tt' (this is the nominal corner) then click **OK**.

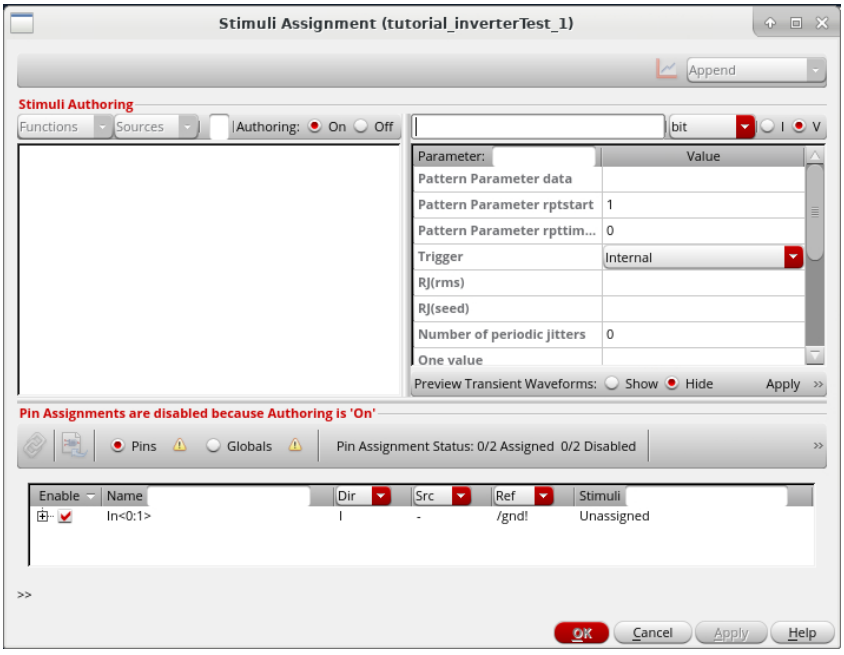


- To setup the type of analysis to be performed (transient, DC, AC etc). Click **Click to add analysis** in the left pane. A new window appears. For this tutorial, we will perform a transient analysis that starts from 0ns to 30ns (Type in '30n' instead of '30ns'). Select *tran* for *Analysis* and enter the corresponding values. Click **OK** when done.
- Now in the *ADE* window, the *Analyses* field should now display the type of analysis to be performed with the corresponding arguments.

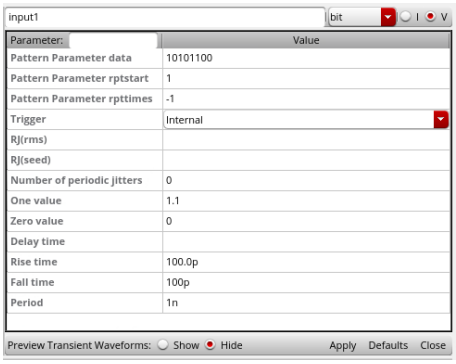
2.3 *Stimuli Setup*

- To setup stimulus or input signals, select **Setup -› Stimuli**.

- A new window named *Stimuli Assignment* appears. The top right section is used to create a new stimulus waveform. The top left section lists all created waveforms. The bottom section is used to assign waveforms to pins.

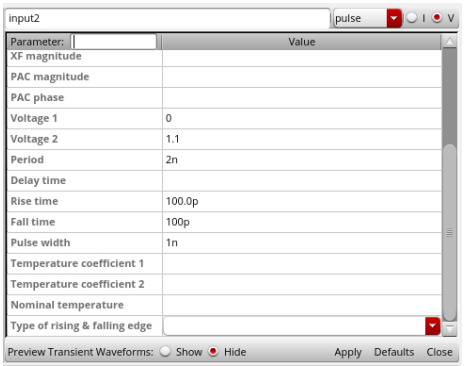


- In the top right section, type ‘input1’ into the text box. Select ‘bit’ from the dropdown, and ensure the button for ‘V’ is selected. Enter the following details:
  - Pattern Parameter Data: 10101100
  - rpttimes: -1 (setting to -1 repeats infinitely)
  - One value: 1.1
  - Zero Value: 0
  - Rise Time: 100p
  - Fall Time: 100p
  - Period: 1n



- Click **Apply**. You should now see ‘input1’ in the top left pane.

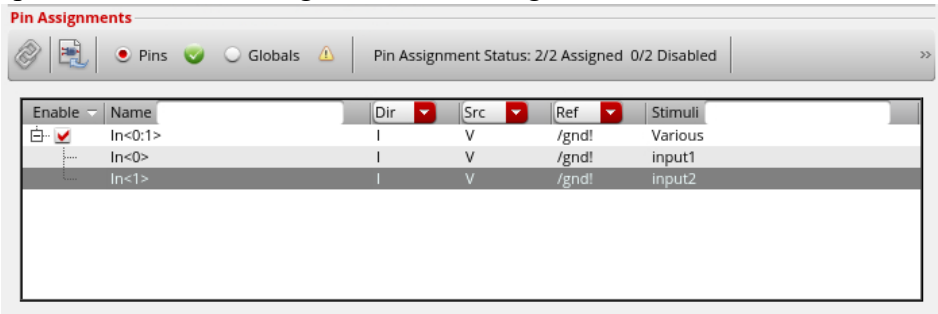
- Change the text in the top box to ‘input2’. Change the dropdown to ‘pulse’ Enter the following parameters:
  - Voltage 1: 0
  - Voltage 2: 1.1
  - Period: 2n
  - Rise time: 100p
  - Fall Time: 100p
  - Pulse Width: 1n



- Click **Apply**

- Change the text in the top box to “supply”. Change the dropdown to ‘DC’. Enter 1.1 in the DC voltage field, then click apply. You should now have 3 stimuli available in the left pane.

- Select ‘input1’ in the top left pane and In<0> in the bottom pane. Click the Assign Stimuli button ( ).
- Select ‘input2’ and In<1> and again click the assign stimuli button.



- Select the radio button for **Globals** then repeat the above process to assign ‘supply’ to ‘vdd!’. The ‘vdd!’ pin is connected to all vdd symbols in your schematic.

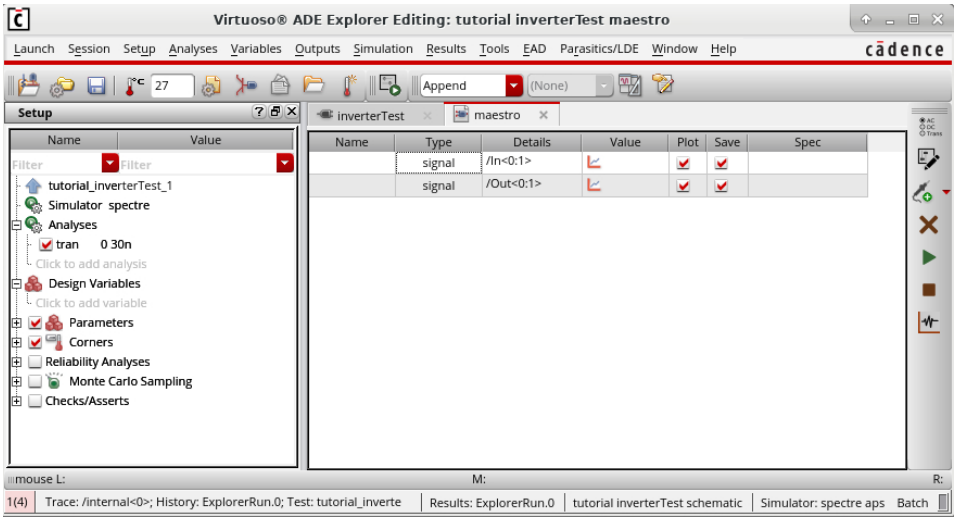
After you have entered all the parameters, Click **OK** to finish setting up stimuli.

2.4 Simulation Output Data

- The data obtained from the simulation can be saved and/or plotted. The saved data will be written to disk in the *Project Directory* specified above. The plotted set of data can be plotted in a Waveform window.

- To add nodes and terminals to the saved or plotted set, select **Outputs -› To Be Plotted -› Select On Schematic**. In the schematic window, choose one or more nodes or terminals. When selecting, click on the square pin symbols to choose currents and click on wires to choose voltages. For this tutorial, select the input, and output wires (bus). Note that if you select a bus, a new window opens called *Select bits from bus*. In this window, select the bits you want to plot for the selected bus and click **OK**. Once you have selected the buses, press **Esc**.

- Select the maestro tab. The input and output busses should be listed in the main window.

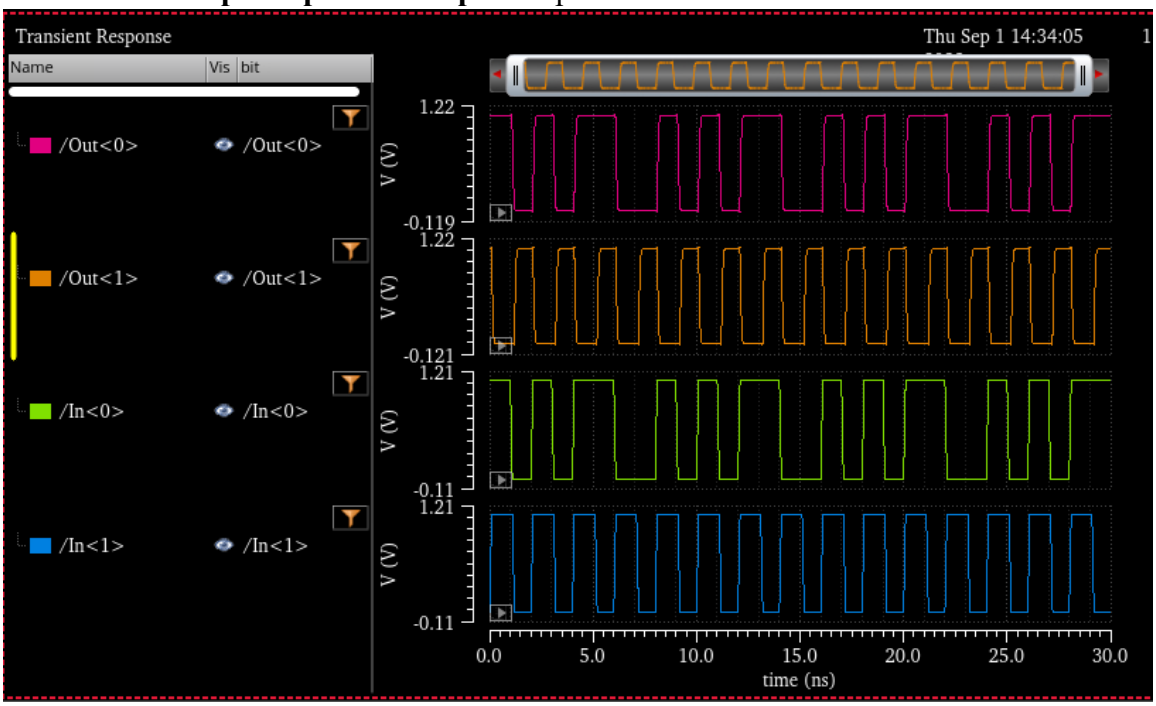


- To delete a node or terminal from the saved or plotted set, right click on it and select *Delete*

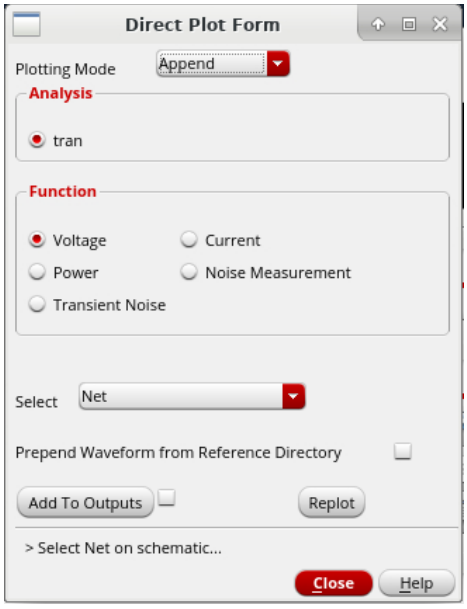
- To run the simulation, select **Simulation -› Netlist and Run** or click the *Netlist and Run* icon on the right. The CIW will display the status and the results of the simulation. Errors or warnings will be displayed here if found. An output log can be found from **Simulation -> Output Log** in the *Analog Design Environment* window.

2.5 *Viewing Simulation Results*

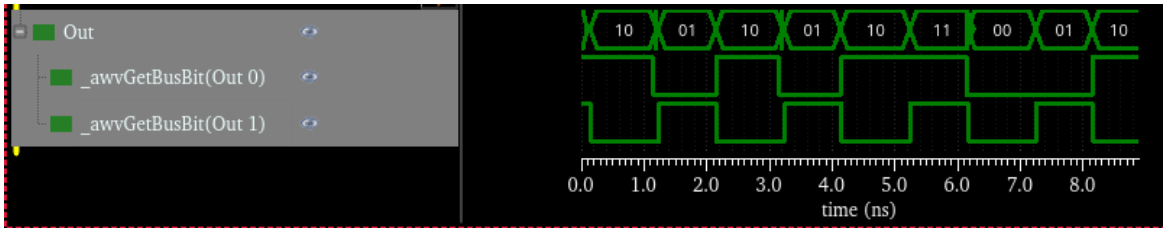
- After a successful simulation, the plotted results will appear as a window inside of ADE. Click the **Launch ViVA** button () to bring the plot into its own window. In order to make the plot easier to interpret, we can split the signals into multiple strips. Right click on a signal then select **Move To -> New Strip**. Repeat until you have four separate strips. You can also use **Graph->Split All Strips** to separate different buses.



- By default, all voltages are saved regardless of your output settings. This means we can still plot outputs we forgot to set up. In the *ADE* window, select select **Results -> Direct Plot -> Main Form**. Two new windows appear, one of them called *Direct Plot Form*. Make sure *Plotting Mode* is *Append* the *Analysis* is *tran*, *Function* is *Voltage* and Select is *Net*. **Do not** click **OK** on this window. Instead, select the nets or buses in the schematic for which you want the voltage transient plot. Note that selecting each wire appends its plot to the same graph. Select the internal bus to plot its waveform.



- The analog traces are useful for measuring things like rise and fall time, but are less useful for determining if a large digital circuit is working correctly. In the plot window, select **Measurement->Analog to Digital.** Left click Out<0> then Out<1>. Enter 0.55 in the center field. Check the box for **Make Bus**. Enter *Out* in the Bus name field. Click **OK**. This converts the analog waveform into a digital waveform that can occasionally be easier to interpret.



2.6 *Saving and Loading States*

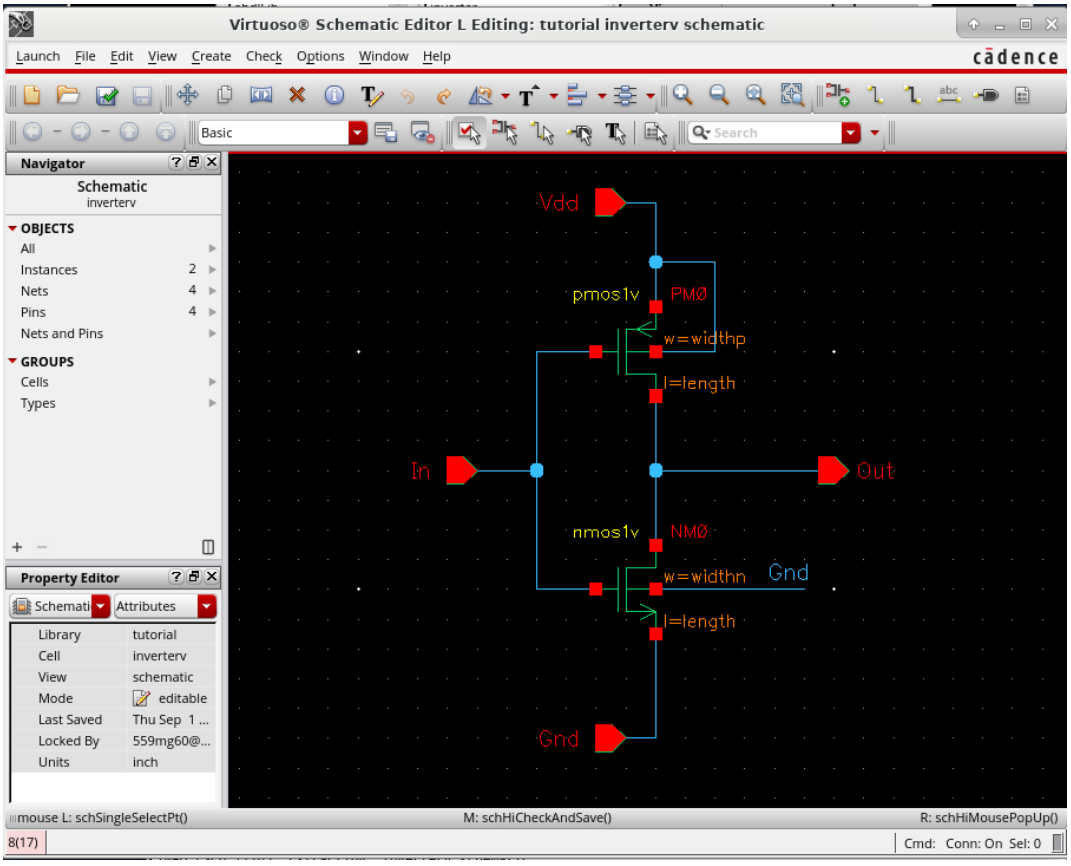
- Saving states allow you to save and restore all or part of the simulation setup so that lot of steps can be saved when setting up the simulation (for a similar simulation run). To do this, select **Session -› Save**. You can also have multiple maestro views with different simulation setups. You can select **Session -> Save a copy** to create a new maestro view.

3 Variable and Parametric Analysis

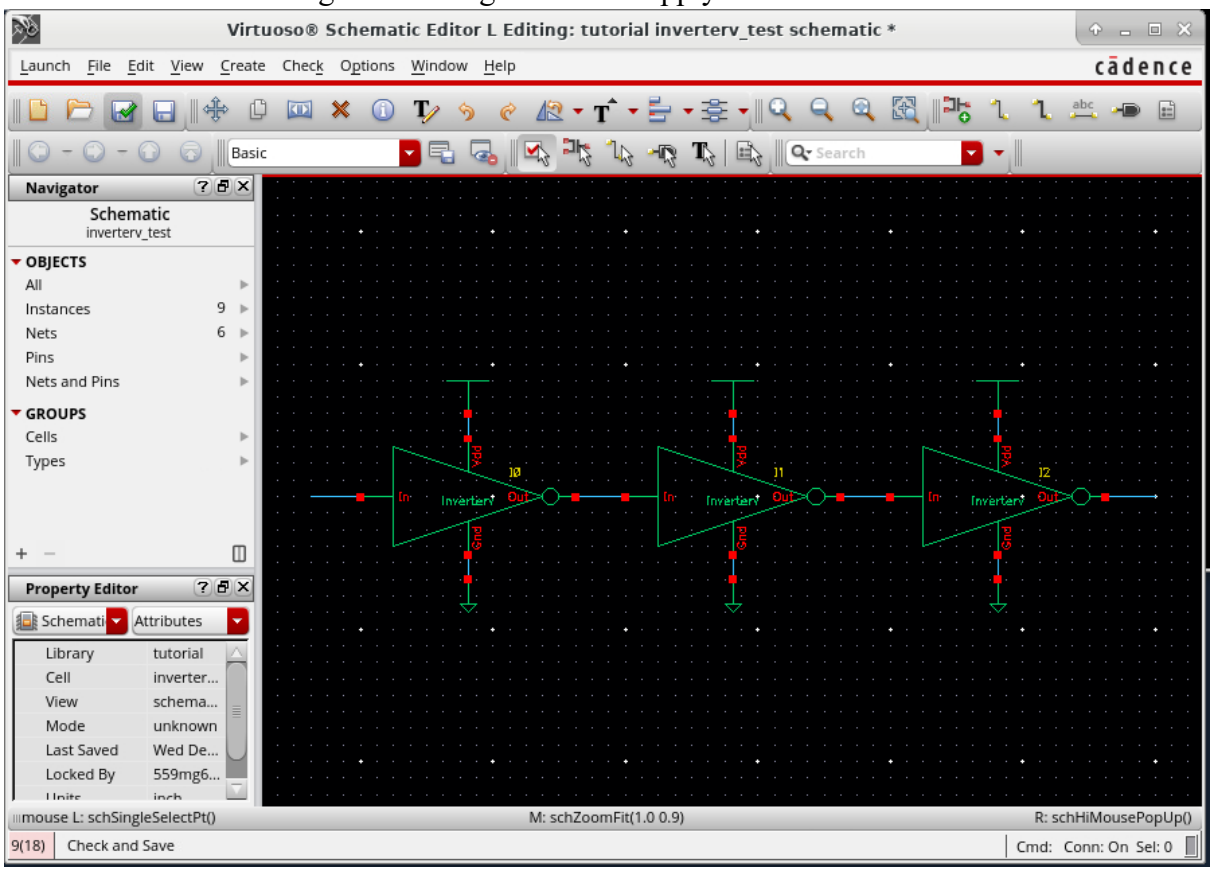
3.1 *Create a cellview*

Go to the library manager. Create a copy of your inverter cellview by right clicking on *inverter* and selecting *copy*. Change the *To* field to *inverterv* then click **OK.** Open the inverterv schematic by selecting inverterv then double clicking on schematic. Change the transistor parameters to the following:

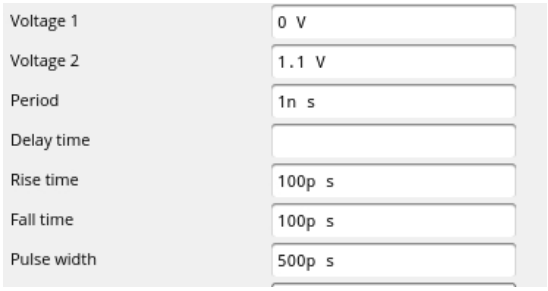
- PMOS total width : widthp
- PMOS length : length
- NMOS total width : widthn
- NMOS length : length



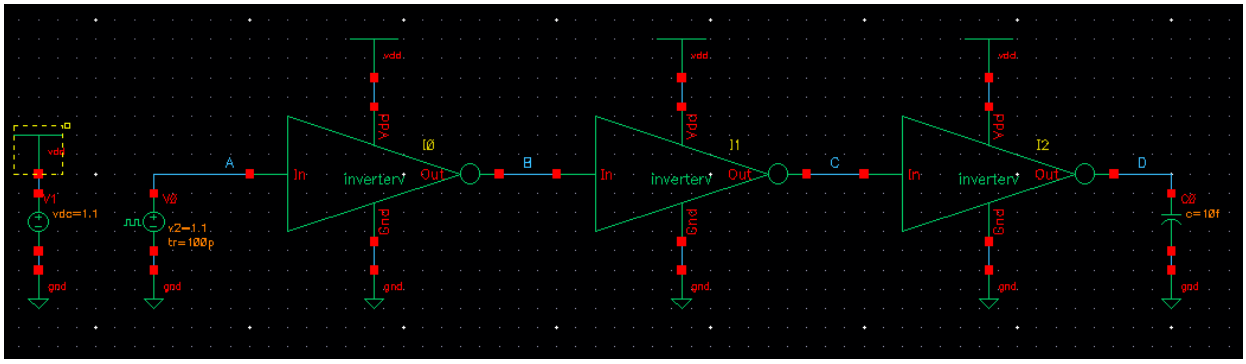
- Save and close the inverterv schematic.
- Create a new schematic called “inverterv_test”
- Place 3 instances of inverterv in the schematic and connect them as shown. Use the *gnd* and *vdd* cells from analogLib for the ground and supply connections.



- Place a *vpulse*, *vdc*, and *cap* from the analogLib into your schematic. Connect the *vdc* block between a *gnd* and *vdd* symbol. Connect the *vpulse* to the input of the first inverter. Connect the *cap* to the output of the last inverter.
- Use the wire name tool (hotkey **l**) to name each node in the schematic A, B, C, and D from left to right.
- Set the value of the capacitor to 10fF (enter 10f as the value).
- Set the DC voltage for *vdc* to 1.1V
- Set the parameters for the *vpulse* to
  - Voltage 1: 0V
  - Voltage 2: 1.1V
  - Period: 1ns
  - Rise Time: 100ps
  - Fall Time: 100ps
  - Pulse Width: 500ps



- The final schematic should look like this:



- This is a different approach to test bench setup than the stimulus method used in section 2.

3.2 Launch Analog Design Environment (ADE)

- Launch Analog Design Environment **Launch -› ADE Explorer**
- Select **Create New View**, click **OK**, then click **OK** again.
- In ADE, **Setup -› Simulator**: select *spectre* for the *Simulator*.
- Now select **Setup -> Model Libraries** from the drop down menu. Change the section to *tt* and then click **OK**.

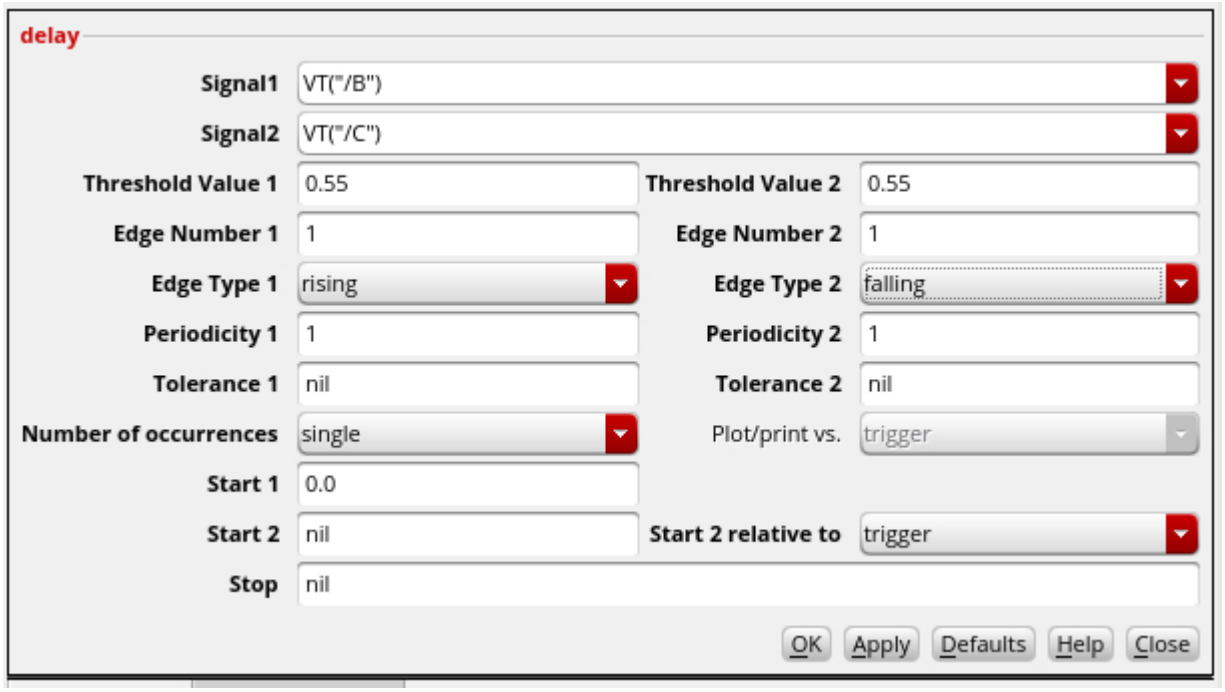
- To setup the type of analysis to be performed (transient, DC, AC etc). Select **Analyses -› Choose** or click the *Choose Analyses* icon on the right. A new window appears. For this tutorial, we will perform a transient analysis that starts from 0ns to 3ns (Type in '3n' instead of '3ns'). Select *tran* for *Analysis* and enter the corresponding values. Click **OK** when done.

- Now in the *Analog Design Environment* window, the *Analyses* field should now display the type of analysis to be performed with the corresponding arguments.

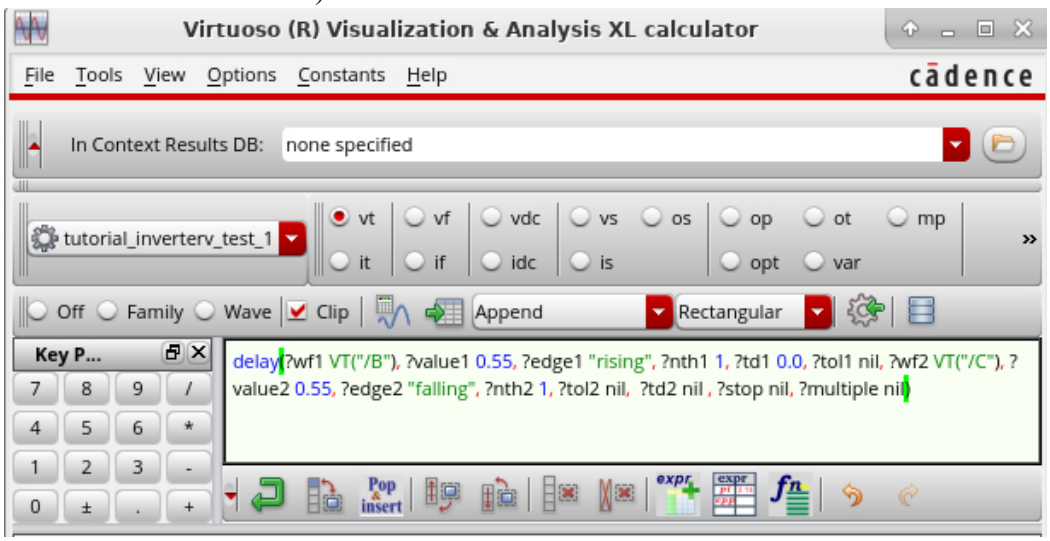
- To add nodes and terminals to the saved or plotted set, select **Outputs -› To Be Plotted -› Select On Schematic**. In the schematic window, choose all four nodes (A,B,C and D). When selecting, click on the square pin symbols to choose currents and click on wires to choose voltages.

3.3 Setting up Additional Outputs for Delay Measurements

- In ADE, **Tools->Calculator**
- Click *vt* button in *Calculator,*
- Click *C* wire in schematic window,
- Click *B* wire in schematic window,
- Select **delay** from the functions in the *Function Panel* on the bottom. The window changes into the figure shown. Please make sure that the **Signal1** is corresponding to *VT("/B")* and **Signal 2** is corresponding to *VT("/C")*. Change the other values as shown.



- Click **OK** and you will see the following screen with the formula depicted. What we have done here is generated an expression to measure the delay from the first rising edge of the input to the first falling edge of the output. The threshold voltage levels have been given as 0.55V for both the input and the output signals (as filled in *Threshold Value 1* and *Threshold Value 2* boxes).



- In *Calculator* window, Click **Tools->Send Buffer to ADE Outputs** or click the  icon.

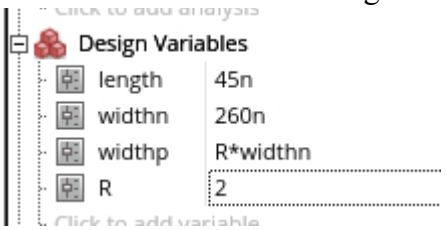
- In the *ADE Explorer* window, set the name of this output to *tphl*

- Similarly, add one more output *tplh* which measures the delay from the 1st falling edge of the input to the 1st rising edge of the output.

- In *Setting Outputs* window, Click **OK.**

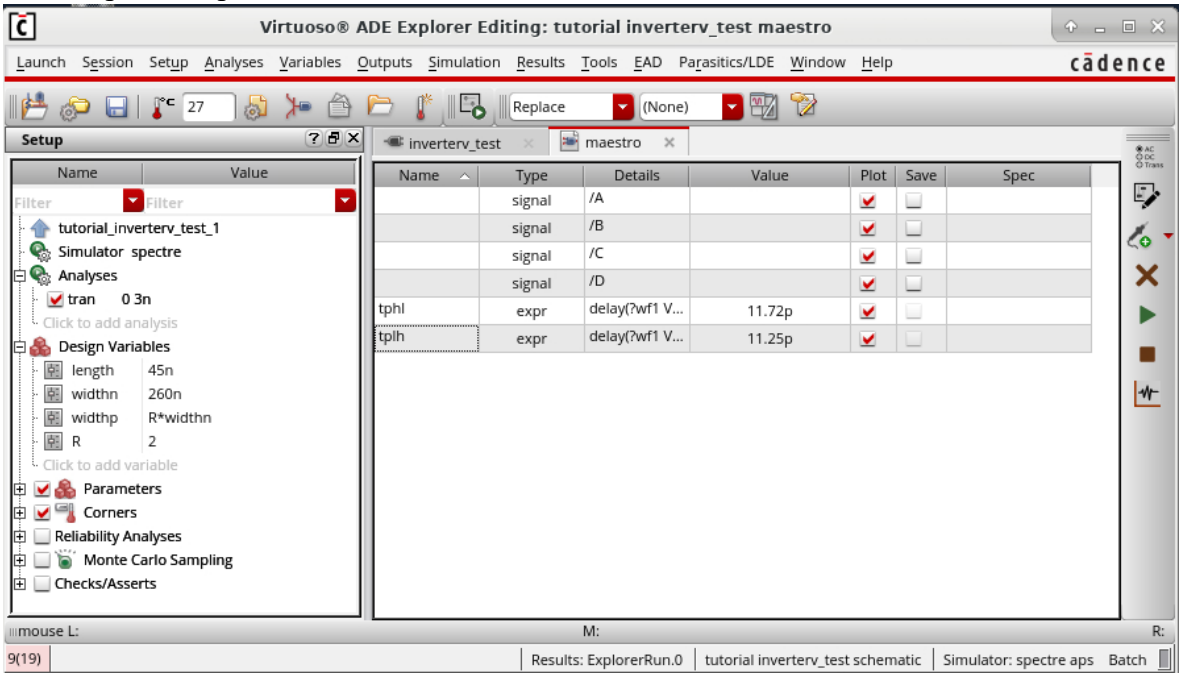
3.4 Variable setup

- In ADE, Right click on **Design Variables** and select **Copy from Cellview**
- Variables will appear in ADE window.
- Click the **click to add variable** area and create a new variable called *R*
- Double click the variables to set the values as the next figure.



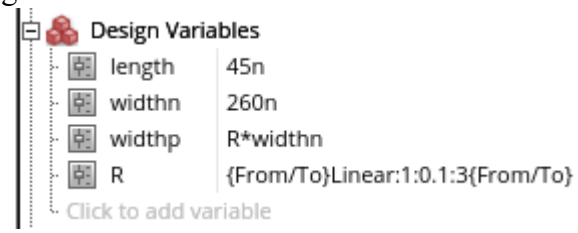
3.5 Running Simulation

- Run the simulation and note that there are delay measurement results beside the output names tphl and tplh.

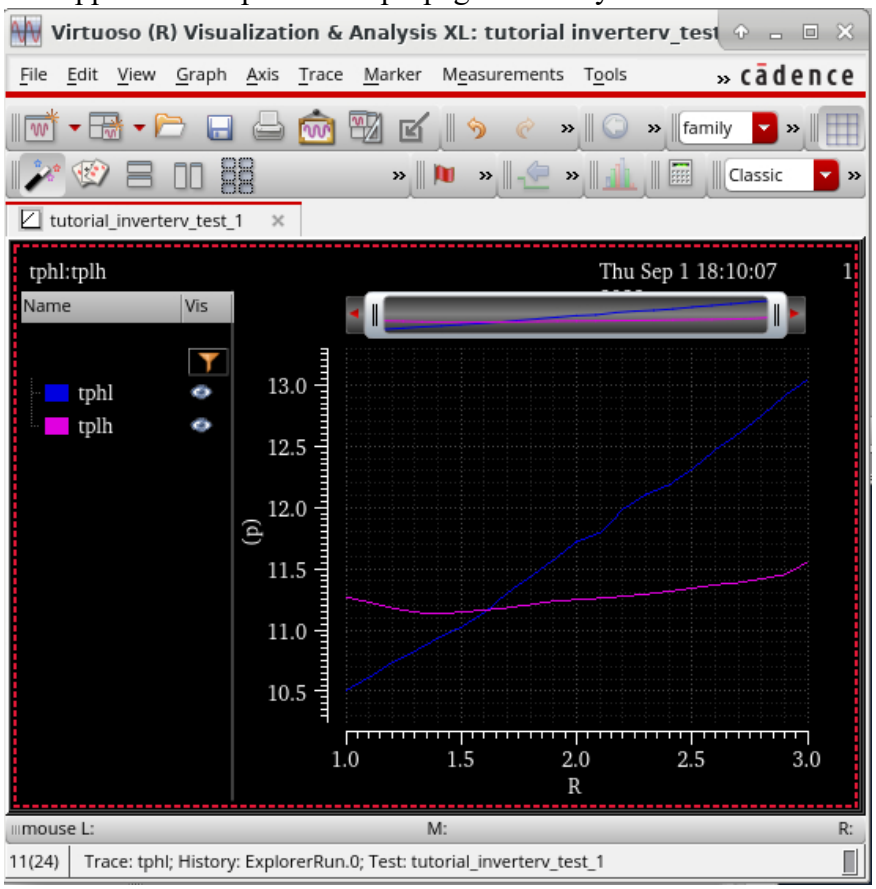


3.6 *Parametric Analysis*

- In the *Design Variables* section, double click on the value of *R* then click the "…" symbol.
- Click **Delete Spec** to remove the 2.
- From the **Add Specification** dropdown, select **From/To**
- Change *Step Type* to *Linear*
- Set the variables to:
  - From: 1
  - To: 3
  - Step Size: 0.1
- Click **OK.** Your design variables should now look like this:



- In the simulation, uncheck the boxes for *plot* for the outputs A,B,C and D.
- In the *ADE* window, select **Setup->Job Setup** and change *Job Control Mode* to ICRP and *Max. Jobs* to 8. Click **OK.**
- Run the simulation.
- A window will appear with a plot of the propagation delay vs. the size ratio of P to N.



4 Spectre Netlist Extraction and Example

4.1 Spectre Netlist Extraction

A netlist file describes the system or circuit to be simulated. The file specifies the components or instances in the circuit, how the circuit is connected together, what type of simulation is to be run, what inputs are supplied and what outputs are to be obtained.

- To extract a Spectre format netlist, the procedures are identical to the ones required to setup a Spectre simulation in *Analog Design Environment*, except that instead of running the simulation, a netlist is extracted. This is because although it is transparent to the user, a netlist is generated before the Spectre simulation is run under the *Analog Design Environment*.

- If a successful simulation run has been done, the netlist is already created and can be readily viewed by selecting **Simulation -› Netlist -› Display**.

- Open the maestro view for the inverterTest cellview

- Select **Simulation -› Netlist -› Create**. The CIW will display the status and the results of the netlist extraction process. Errors or warnings will be displayed here if found.

- If the netlist extraction is successful, a new window displaying the extracted netlist appears. Save the netlist doing **File->Save As** and name the netlist inverterTest.scs

4.2 Spectre Netlist Example

Here is an example that shows how a Spectre netlist could look like.

```
// Generated for: spectre
// Generated on: Nov  8 13:55:42 2018
// Design library name: tutorial
// Design cell name: InverterTest
// Design view name: schematic
simulator lang=spectre global 0
vdd!
include "/package/eda/cells/ncsu-cdk-
1.6.0.beta/models/hspice/public/tsmc18dN.m" include
"/package/eda/cells/ncsu-cdk-
1.6.0.beta/models/hspice/public/tsmc18dP.m"

// Library name: tutorial
// Cell name: inverterp // View
name: schematic subckt
inverterp Gnd In Out Vdd
parameters wn=2.7e-07 ln=1.8e-07 wp=5.4e-07 lp=1.8e-07
    N0 (Out In Gnd Gnd) tsmc18dN w=wn l=ln as=wn * 2.5 * (180.0n) ad=wn * 2.5 *
(180.0n) \
         ps=(2 * wn) + (5 * (180.0n)) pd=(2 * wn) + (5 * (180.0n)) m=1 \
region=sat
    P0 (Out In Vdd Vdd) tsmc18dP w=wp l=lp as=wp * 2.5 * (180.0n) ad=wp * 2.5 *
(180.0n) \
         ps=(2 * wp) + (5 * (180.0n)) pd=(2 * wp) + (5 * (180.0n)) m=1 \
region=sat ends inverterp
// End of subcircuit definition.

// Library name: tutorial
// Cell name: InverterTest
// View name: schematic
I4 (0 internal\<1\> Out\<1\> vdd!) inverterp wn=2.7e-07 ln=1.8e-07 \
wp=5.4e-07 lp=1.8e-07
I3 (0 internal\<0\> Out\<0\> vdd!) inverterp wn=2.7e-07 ln=1.8e-07 \
wp=5.4e-07 lp=1.8e-07
I2 (0 net23 internal\<1\> vdd!) inverterp wn=2.7e-07 ln=1.8e-07 wp=5.4e-07 \
lp=1.8e-07
I1 (0 In\<1\> net23 vdd!) inverterp wn=2.7e-07 ln=1.8e-07 wp=5.4e-07 \
lp=1.8e-07
I0 (0 In\<0\> internal\<0\> vdd!) inverterp wn=2.7e-07 ln=1.8e-07 \
wp=5.4e-07 lp=1.8e-07 C1 (Out\<1\> 0) capacitor c=10f m=1 C0
(Out\<0\> 0) capacitor c=10f m=1 include "./_graphical_stimuli.scs"
simulatorOptions options reltol=1e-3 vabstol=1e-6 iabstol=1e-12 temp=27 \
tnom=27 scalem=1.0 scale=1.0 gmin=1e-12 rforce=1 maxnotes=5 maxwarns=5 \
digits=5 cols=80 pivrel=1e-3 sensfile="../psf/sens.output" \
checklimitdest=psf
tran tran stop=30n write="spectre.ic" writefinal="spectre.fc" \
annotate=status maxiters=5
finalTimeOP info what=oppoint where=rawfile modelParameter
info what=models where=rawfile
```

5 Vector Files and Spectre X MS

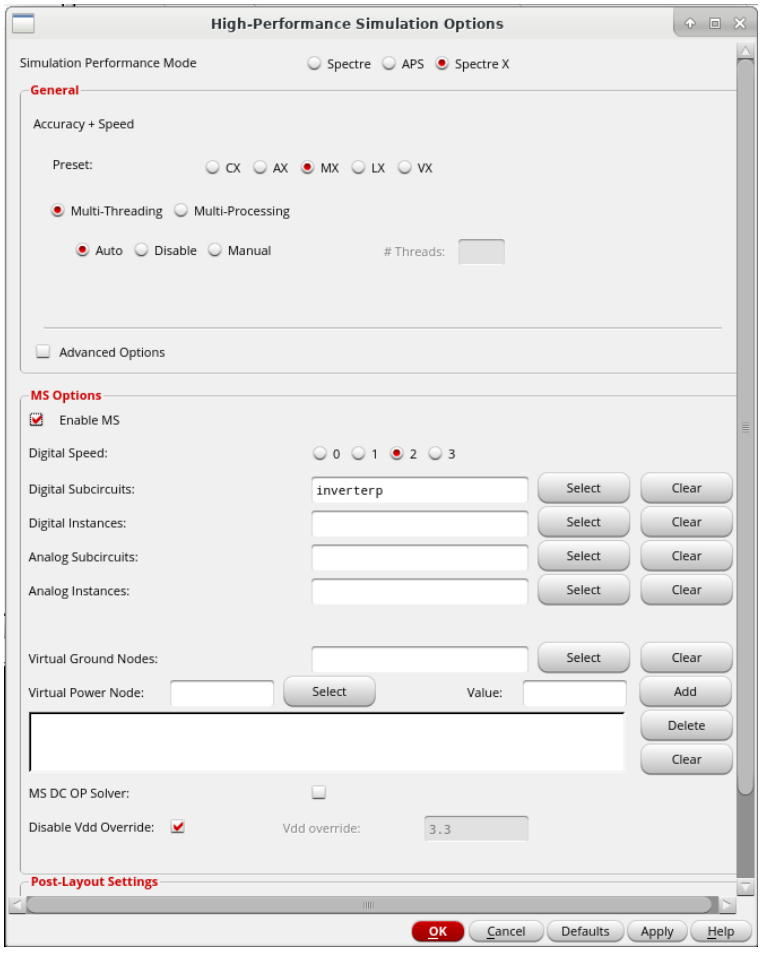
As the circuits you design get more complicated, simulation setup complexity and run times start to grow exponentially, especially when doing full transistor level simulation of digital blocks. In order to speed up the verification process, we turn to using test vectors (rather than manually configuring individual bits in the stimulus editor) and high performance multithreaded fastSPICE simulators.

5.1 Spectre X MS

Spectre X MS (Formerly Spectre XPS MS) is a transistor-level circuit simulator that can run simulations (with Spectre-like accuracy) with a run time 10 or more times faster than Spectre. Spectre X MS partitions a design in to analog and digital parts and uses fastSPICE simulation for the digital parts and normal simulation for the analog parts. This would be very helpful especially when the design complexity increases as these simulations may take hours.

Simulating with Spectre X MS can be configured easily from *ADE*.

- From the *ADE* window for *inverterTest*, select **Setup->High Performance Simulation** then select Spectre X. In the Preset field, you can choose a tradeoff between accuracy and speed. 'CX' is the most accurate, but slowest. 'VX' is the least accurate, but fastest.
- Using Spectre X will already accelerate the simulation of a complicated design, as it performs optimal multicore simulation of your design. We can increase the speed even further for digital designs by turning on the mixed signal options. Check the box for *Enable MS*
- In order to tell Spectre X which circuits are digital, click *Select* next to the *Digital Subcircuits* field. This will take you to the schematic. Select one of the inverters then click ESC.
- You should see *inverterp* in the digital subcircuits field. The simulator will use digital optimization anywhere you use an inverterp block. For any instances not explicitly marked in the MS options, the simulator will try to guess if it should be analog or digital.



- Click **OK**. We won't see a significant change in run time for such a simple design, but you should consider using this approach for the much more complicated circuits you will simulate in the future.

5.2 Using Input Vector File

When testing more complicated designs, it is often desirable to set up vector files to allow for more in depth pattern generation than is offered by the stimulus editor. Additionally, vector files can be used to validate expected results. Full documentation on vector file commands is available by searching for *Digital Vector File Format* in the **Help** dropdown menu.

This example shows how to use a vector file in your simulation. Create a file named `input.vec` as below.

```
; Vector File Comment

radix 11

vname In<[1:0]>

io ii

vih 1.1
vil 0.0

slope 0.01

0 00
10 01
15 10
20 11
```

Each statement is briefly described below.

- ***radix*** – The *radix* statement must be the first non-comment line. It specifies the number of bits associated with each table vector column. In the above case, radix *11* defines two 1-bit vectors, In<1> and In<0>.

- ***vname*** – The *vname* statement defines the name of each vector.

- ***io*** – The *io* statement defines the type of each vector. *i* defines input, *o* defines output, *b* defines bi-direction, and *u* defines unused vector.

- ***vih*** – Specifies the logic-high voltage for input or bidirectional vectors when in input mode.

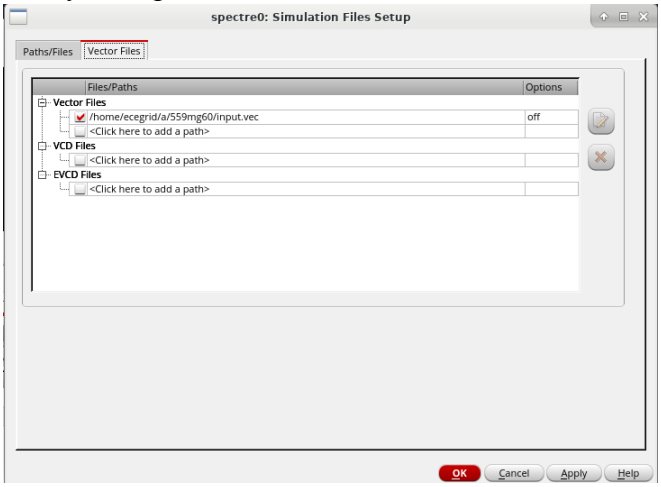
- ***vil*** – Specifies the logic-low voltage for input or bidirectional vectors when in input mode.

- ***slope*** – Defines the rise and fall times for the input vectors to transition from logic-low voltage (*vil*) to logic-high voltage (*vih*) or from logic-high voltage (*vih*) to logic-low voltage (*vil*).

- In the tabular data section, the first column lists the absolute time. The second and subsequent columns list the signal logic states at the absolute time.

Follow the steps below.

- In the *ADE* window, select **Setup->Simulation Files**
- Switch to the *Vector Files* tab.
- Add your *input.vec* file into the *Vector Files* section



- Click **OK**
- Open **Setup->Stimuli.** Uncheck *enable* for the two input signals. Make sure that *enable* is checked for the global vdd! Input. Click **OK**.
- Run the simulation. You should see that the inputs are set according to the vector file.

6 NanoTime

Synopsys NanoTime is a full-chip, transistor-level critical path finder and static timing analysis solution for custom circuit designs. It employs static timing analysis techniques to enable you to detect and correct timing violations. It searches all possible paths in your design, calculates delay, and checks timing requirements at all the timing nodes. It is a replacement of the previous tool PathMill. NanoTime is the next-generation transistor-level static timing analysis solution that addresses the emerging challenges in signal integrity (SI) analysis associated with custom designs. NanoTime offers concurrent timing and SI analysis, accuracy within five percent of HSPICE and Spectre. Its integration with *PrimeTime* (that is a gate-level critical path finder and static timing analysis tool) enables full-chip analysis of designs that includes both gate- and transistor-level blocks.

6.1 Using Nanotime

In order to run nanotime, you must run `module load synopsys/nanotime` in the shell you want to run nanotime. If you are using nanotime frequently, you can add this command to your .bashrc file.

The *user guide* for `nanotime` can be viewed by entering `ntdoc` at a terminal window.

6.2 SPICE Netlist

We will generate a SPICE netlist of our *InverterTest* schematic and will make a few modifications to it to allow us to run NanoTime.

- In the *CIW* window, click on **File -> Export -> CDL**. A window called *Virtuoso CDL Out* appears.

- In the *Output CDL Netlist File* field, enter `nanotime.sp`. Change the *Netlisting Mode* from **Digital** to **Analog**. Check the box for "Renetlist". Leave everything else as default and click **OK**. After a few seconds, a window will pop up saying the netlist creation has succeeded.

- You should now have a SPICE netlist called `nanotime.sp` which looks like as below.

```
********************************************************************************
* auCdl Netlist:
*
* Library Name:  tutorial
* Top Cell Name: inverterTest
* View Name:     schematic
* Netlisted on:  Sep  1 18:52:41 2022
********************************************************************************
*.BIPOLAR
*.RESI = 2000
*.RESVAL
*.CAPVAL
*.DIOPERI
*.DIOAREA
*.EQUATION
*.SCALE METER
*.MEGA
.PARAM

*.GLOBAL vdd!
+        gnd!

*.PIN vdd!
*+    gnd!

********************************************************************************
* Library Name: tutorial
* Cell Name:    inverterp
* View Name:    schematic
********************************************************************************
.SUBCKT inverterp Gnd In Out Vdd
*.PININFO Gnd:I In:I Vdd:I Out:O
MNM0 Out In Gnd Gnd g45n1svt m=1 l=ln w=wn
MPM0 Out In Vdd Vdd g45p1svt m=1 l=lp w=wp
.ENDS

********************************************************************************
* Library Name: tutorial
* Cell Name:    inverterTest
* View Name:    schematic
********************************************************************************

.SUBCKT inverterTest In<0> In<1> Out<0> Out<1>
*.PININFO In<0>:I In<1>:I Out<0>:O Out<1>:O
XI4 gnd! internal<0> Out<0> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI3 gnd! internal<1> Out<1> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI2 gnd! net6 internal<1> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI1 gnd! In<1> net6 vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI0 gnd! In<0> internal<0> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
CC1 Out<0> gnd! 10f $[CP]
CC0 Out<1> gnd! 10f $[CP]
.ENDS
```

- Create a folder called `nanotime` either in your Home directory or on your Desktop, and paste this netlist there.
- Now make the following modifications to `nanotime.sp`
    - Delete the initial few lines starting from `*.BIPOLAR` and ending at `.PIN`
    - In the `inverterp` subcircuit, change `g45n1svt` in the `MNM0` line to `NMOS_VTL` and change `g45p1svt` in the `MPM0` line to `PMOS_VTL`
- The modified netlist should look like this:

```
********************************************************************************
* auCdl Netlist:
*
* Library Name:  tutorial
* Top Cell Name: inverterTest
* View Name:     schematic
* Netlisted on:  Sep  1 18:58:02 2022
********************************************************************************


*.PIN vdd!
*+    gnd!


********************************************************************************
* Library Name: tutorial
* Cell Name:    inverterp
* View Name:    schematic
********************************************************************************

.SUBCKT inverterp Gnd In Out Vdd
*.PININFO Gnd:I In:I Vdd:I Out:O
MNM0 Out In Gnd Gnd NMOS_VTL m=1 l=ln w=wn
MPM0 Out In Vdd Vdd PMOS_VTL m=1 l=lp w=wp
.ENDS

********************************************************************************
* Library Name: tutorial
* Cell Name:    inverterTest
* View Name:    schematic
********************************************************************************

.SUBCKT inverterTest In<0> In<1> Out<0> Out<1>
*.PININFO In<0>:I In<1>:I Out<0>:O Out<1>:O
XI4 gnd! internal<0> Out<0> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI3 gnd! internal<1> Out<1> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI2 gnd! net6 internal<1> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI1 gnd! In<1> net6 vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
XI0 gnd! In<0> internal<0> vdd! / inverterp wp=3.9e-07 lp=4.5e-08 wn=2.6e-07
+ ln=4.5e-08
CC1 Out<0> gnd! 10f $[CP]
CC0 Out<1> gnd! 10f $[CP]
.ENDS
```

6.3 Tech File

Create a file `tech.sp` with the contents as follows.

```
*nanosim tech="direct"
*nanosim tech="voltage 1.10"
*nanosim tech="vds 0 1.10 0.02"  *nanosim tech="vgs 0 1.10 0.02"
.include
"/package/eda/cells/FreePDK45/models/hspice/tran_models/models_no
m/NMOS_VTL.inc"
.include
"/package/eda/cells/FreePDK45/models/hspice/tran_models/models_no
m/PMOS_VTL.inc"
```

The SPICE model file might contain directives to specify how NanoTime will expand the model to make the lookup tables. The `direct` directive is an optional header to indicate that the following lines are directives for generating the transistor lookup tables.

- `*nanosim tech= "voltage VDD"` NanoTime uses the voltage information to calculate transistor capacitance values used in simulation. In the absence of a voltage directive, NanoTime uses the largest of the V_{dd} voltages specified in the netlist, if available. Otherwise, it uses the voltage specified by the `oc_global_voltage` variable (to be introduced in the next section).

- The `vds` and `vgs` directives specify the end voltage and the voltage step size for the sweeps used to generate the lookup tables for the drain-to-source, gate-to-source, and body-to-source voltage tables, respectively. You might want to specify a finer resolution to minimize interpolation error, or a larger end voltage to accommodate a larger voltage swing, at the cost of more simulation time.

- The .include statements load the spice models. Note that we are using models from FreePDK45 instead of GPDK45, as GPDK45 does not include models compatible with NanoTime.

6.4 Script/Configuration File

Create a file `nanotime.scr` with the contents as follows.

```
set search_path {.}
set library_path {.}
set link_path {*}

set oc_global_voltage 1.1

register_netlist -format spice {nanotime.sp tech.sp}

link_design inverterTest

report_port

set input_ports {In<1> In<0>}
set_port_direction -input $input_ports
set output_ports {Out<1> Out<0>}
set_port_direction -output $output_ports

report_port

report_design
report_net
report_cell

create_clock -name MCLK -period 10.0
report_clock

match_topology
report_transistor_direction
report_topology
check_topology

set_input_delay -clock MCLK -rise 1.0 {In<1> In<0>}
set_input_delay -clock MCLK -fall 1.0 {In<1> In<0>}
set_input_delay -clock MCLK -add_delay 1.0 {In<1> In<0>}
report_port -input_delay

set_output_delay -clock MCLK 0.0 {Out<1> Out<0>}
report_port -output_delay

check_design
trace_paths
report_paths -max -max_paths 10
exit
```

- NanoTime searches for each file in each directory listed in the `search_path` variable and uses the first one found.

- `library_path` contains a list of directories to locate undefined sub-circuit references, for compatibility with PathMill.

- `link_path` specifies the location of a list of libraries, design files, and library files used during linking.

- `oc_global_voltage` specifies the default power supply voltage.

- The `match_topology` command runs the NanoTime topology recognition algorithm. NanoTime examines the configuration of transistors in the design to identify and mark various circuit structures such as latches, inverters, and multiplexers. NanoTime needs this information so that it can perform timing checks at the appropriate structures in the design.

- `check_topology` checks for errors in topologies.

- `set_input_delay` specifies the data arrival time at an input relative to a clock.

- `set_output_delay` specifies the data required time at an output relative to a clock.

- The `check_design` command divides the design into units called channel connected blocks (CCBs). Each CCB is a set of transistors whose sources and drains are connected together in a network, in series or in parallel, or both. NanoTime checks the CCBs and reports any problems it has in interpreting their configuration.

- `trace_paths` traces timing paths in the current design and saves the worst-case paths into a database for reporting and other processing.

- `report_paths` reports the timing of paths found during path tracing. You can use `-path_type full` option to report the complete path.

6.5 Command

```
module_load synopsys/nanotime
nt_shell -f nanotime.scr
```

You can type `quit` to get out from the command prompt `nt_shell>`. Another way is to type `nt_shell` at the UNIX command prompt and then write `source nanotime.scr` at the `nt_shell` command prompt.

6.6 Results

The simulation result reports the delay from source nodes to sink nodes specified in script file. At the end you should get an output similar kind of as shown below. Note that the path delays are in `ns`.

```
                 Path     Path
   Slack    Delay    Type    Startpoint                    Endpoint
--------- -------- ------- -- ------------------------------ -- ------------------------------
   8.945    0.055    D-O f  In<1>                          r  Out<1>
   8.957    0.043    D-O r  In<0>                          r  Out<0>
   8.961    0.039    D-O r  In<1>                          f  Out<1>
   8.963    0.037    D-O f  In<0>                          f  Out<0>
```

6.7 Worst Case Input Vector

In the above example we have seen that NanoTime provides us paths (having a start-point and an end-point) with rising/falling transitions along the paths. However, to activate the path we need to set some logic values to the other inputs, not only the input from where the path originates.

For example, consider an OR gate. A rising or falling transition from one input will propagate to the output, only if the other inputs are set as *non-controlling* values, i.e., *logic 0* for an OR gate.

NanoTime can write down a SPICE file for simulating a timing path. The command is as follows. Include this command right after `report_paths` command and right before `exit`

```
write_spice -output maxpath.sp -header tech.sp [get_timing_paths -max -max_paths 1]
```

In the generated `maxpath.sp` file, NanoTime writes down the SPICE netlist for a signal transition path. You may extract a worst case primary input vector from the signal-transition path. If you choose to get more than one timing paths in the `get_timing_paths` command, NanoTime will generate multiple output files each for one timing path.